UNIT 1

Introduction: An overview of database management system

5. Course Content and Unit-wise Syllabus

Unit No.	Title & Content	Teaching Hours	Alig
I	Introduction: An overview of database management system, database system Vs file system, Database system concept and architecture, applications of DBMS, data models, schema and instances, data independence, database users, database Administrator(DBA). Data Modeling using the Entity Relationship Model: ER model concepts, notation for ER diagram, attributes, entity sets, relationship, mapping constraints, cardinality, participation constraints, advance ER features - generalisation, specialization, aggregation, conversion from ER diagram to relational schema, Conceptual Design with relevant Examples.	13	CO1

By

Ms. Sonali Uday Mahajan

UNIT 1

Introduction: An overview of database management system

DBMS (Database Management System) is a software system that manages, stores, and retrieves data efficiently in a structured format.

- It allows users to create, update, and query databases efficiently.
- Ensures data integrity, consistency, and security across multiple users and applications.
- Reduces data redundancy and inconsistency through centralized control.
- Supports concurrent data access, transaction management, and automatic backups

Problems with Traditional File-Based Systems

Before the introduction of modern DBMS, data was managed using basic file systems on hard drives. While this approach allowed users to store, retrieve and update files as needed, it came with numerous challenges

- **Data Redundancy:** Duplicate entries across files
- **Inconsistency:** Conflicting or outdated information
- **Difficult Access:** Manual file search required
- **Poor Security:** No control over data access
- **Single-User Access:** No support for collaboration
- **No Backup/Recovery:** Data loss was often permanent

Components of DBMS Applications

1. **Hardware**

- Physical devices like servers, disks, input-output devices (keyboard, monitor, printer).
- Stores and processes data; interfaces between real-world inputs and digital systems.
- Examples: Personal computer hard disk, RAM, network devices used for DBMS operations.

2. **Software**

- Actual DBMS software like MySQL, Oracle, PostgreSQL.
- Includes the database engine, OS, network software, and application tools.
- Translates database access languages into operations.

3. **Data**

- Raw facts stored in structured or unstructured formats.
- **Operational Data**: Actual user data (e.g., name, age).
- **Metadata**: Data about data (e.g., storage time, size, data type).

- Core reason DBMS exists—to manage and store data efficiently.

4. **Procedures**

- Instructions and rules for using DBMS effectively.
- Covers setup, login/logout, data validation, backup, access control, and report generation.
- Helps ensure consistent and secure use of the system.

5. **Database Access Language**

- Used to interact with the database (create, read, update, delete data).
- Examples: SQL, MyAccess, Oracle PL/SQL.
- **DDL (Data Definition Language)** – CREATE, ALTER, DROP
- **DML (Data Manipulation Language)** – INSERT, UPDATE, DELETE

6. **People**

- Users interacting with DBMS at different levels:
- **Database Administrators (DBA)** – Manage security, performance, user access.
- **Developers** – Build applications using the database.
- **End Users** – Use applications to access the database (e.g., students, employees).

Types of DBMS (Data model)

There are several types of Database Management Systems (DBMS), each tailored to different data structures, scalability requirements and application needs. The most common types are as follows:

1. **Relational Database Management System (RDBMS)**

- It organizes data into tables (relations) composed of rows and columns.
- Uses primary keys to uniquely identify rows and foreign keys to establish relationships between tables.
- Queries are written in **SQL (Structured Query Language)**, which allows for efficient data manipulation and retrieval.

Examples: MySQL oracle, Microsoft SQL Server and Postgre SQL.

2. **NoSQL DBMS**

- They are designed to handle large-scale data and provide high performance for scenarios where relational models might be restrictive.
- They store data in various non-relational formats, such as key-value pairs, documents, graphs or columns.
- These flexible data models enable rapid scaling and are well-suited for unstructured or semi-structured data.

Examples: MongoDB, Cassandra, DynamoDB and Redis.

3. **Object-Oriented DBMS (OODBMS)**

- It integrates object-oriented programming concepts into the database environment, allowing data to be stored as objects.
- Supports complex data types and relationships, making it ideal for applications requiring advanced data modeling and real-world simulations.

Examples: ObjectDB, db4o.

4. **Hierarchical Database**

- Organizes data in a tree-like structure, where each record (node) has a single parent and have multiple children.
- This model is similar to a file system with folders and subfolders.
- It is efficient for storing data with a clear hierarchy, such as organizational charts or file directories.
- Navigation is fast and predictable due to the fixed structure.
- It lacks flexibility and difficult to restructure or handle complex many-to-many relationships.

Example: IBM Information Management System (IMS).

5**. Network Database**

- It uses a graph-like model to allow more complex relationships between entities.
- Unlike the hierarchical model, it permits each child to have multiple parents, enabling many-to-many relationships.
- Data is represented using records and sets, where sets define the relationships.
- It is more flexible than the hierarchical model and better suited for applications with complex data linkages.

Example: Integrated Data Store (IDS), TurboIMAGE.

6. **Cloud-Based Database**

- They are hosted on cloud computing platforms like AWS, Azure or Google Cloud.
- They offer on-demand scalability, high availability, automatic backups and remote accessibility.
- These databases can be relational (SQL) or non-relational (NoSQL) and are maintained by cloud service providers, reducing administrative overhead.
- They support modern application requirements, including distributed access and real-time analytics.

Example: Amazon RDS (for SQL), MongoDB Atlas (for NoSQL), Google BigQuery.

Applications of DBMS

- **Banking**: Manages accounts and transactions.
- **E-commerce**: Tracks products, orders, and customers.
- **Healthcare**: Stores patient records and diagnoses.

- **Education**: Handles student grades and schedules.
- **Social Media**: Manages user profiles and interactions.
- **Data Science**: Supports analytics and predictions.

Database system Vs File system (imp)

Basics	File System	DBMS
Structure	The file system is a way of arranging the files in a storage medium within a computer.	DBMS is software for managing the database.
Data Redundancy	Redundant data can be present in a file system.	In DBMS there is no redundant data.
Backup and Recovery	It doesn't provide Inbuilt mechanism for backup and recovery of data if it is lost.	It provides in house tools for backup and recovery of data even if it is lost.
Query processing	There is no efficient query processing in the file system.	Efficient query processing is there in DBMS.
Consistency	There is less data consistency in the file system.	There is more data consistency because of the process of normalization.
Complexity	It is less complex as compared to DBMS.	It has more complexity in handling as compared to the file system.
Security Constraints	File systems provide less security in comparison to DBMS.	DBMS has more security mechanisms as compared to file systems.
Cost	It is less expensive than DBMS.	It has a comparatively higher cost than a file system.
Data Independence	There is no data independence.	In DBMS data independence exists, mainly of two types: 1) Logical Data Independence . 2)Physical Data Independence.
User Access	Only one user can access data at a time.	Multiple users can access data at a time.
Meaning	The users are not required to write procedures.	The user has to write procedures for managing databases

Basics	File System	DBMS
Sharing	Data is distributed in many files. So, it is not easy to share data.	Due to centralized nature data sharing is easy
Data Abstraction	It give details of storage and representation of data	It hides the internal details of Database
Integrity Constraints	Integrity Constraints are difficult to implement	Integrity constraints are easy to implement
Attribute's	To access data in a file, user requires attributes such as file name, file location.	No such attributes are required.
Example	Cobol , C++	Oracle , SQL Server

Database system concept and architecture

A DBMS architecture defines how users interact with the database to read, write, or update information. A well-designed architecture and schema (a blueprint detailing tables, fields and relationships) ensure data consistency, improve performance and keep data secure.

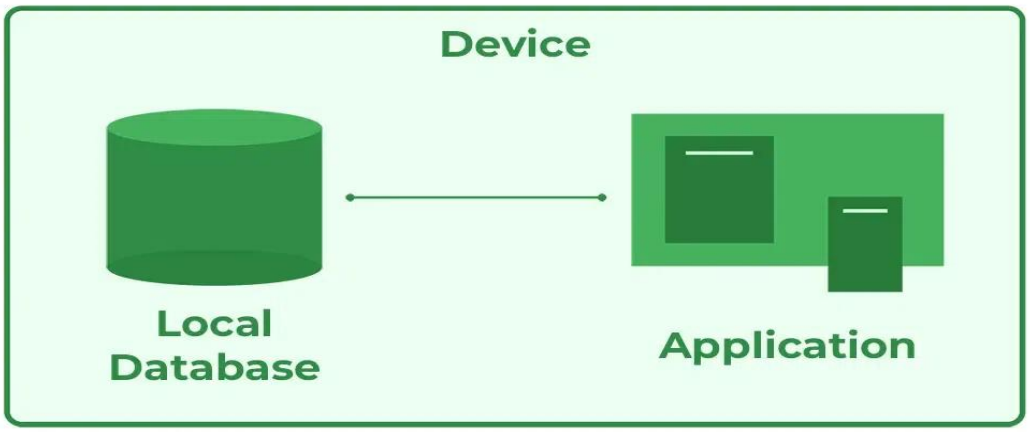
Types of DBMS Architecture

There are several types of DBMS Architecture that we use according to the usage requirements.

- 1-Tier Architecture
- 2-Tier Architecture
- 3-Tier Architecture

1-Tier Architecture

In 1-Tier Architecture, the user works directly with the database on the same system.



- A common example is Microsoft Excel.
- This setup is simple and easy to use,
- This architecture is simple and works well for personal, standalone applications where no external server or network connection is needed.

Advantages of 1-Tier Architecture

Below mentioned are the advantages of 1-Tier Architecture.

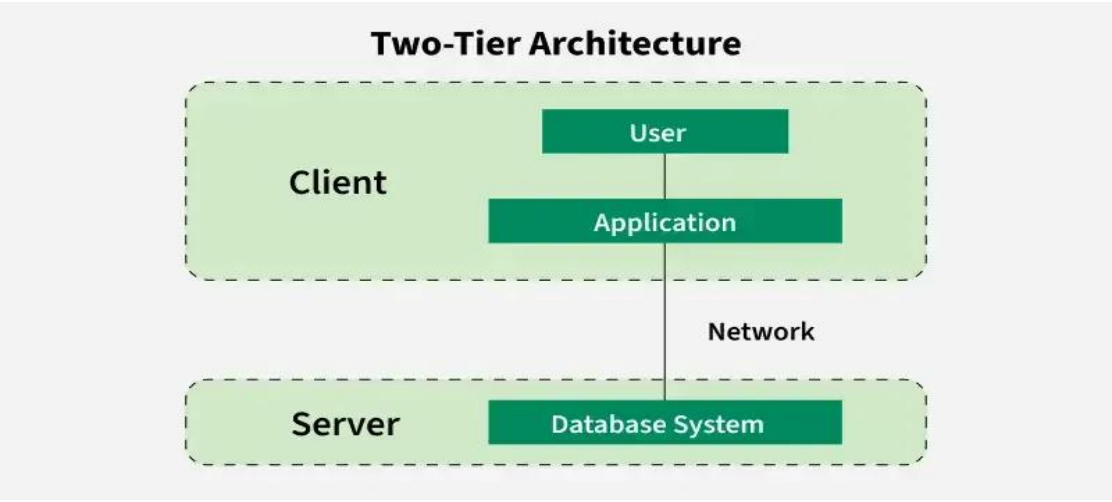
- **Simple Architecture:** 1-Tier Architecture is the most simple architecture to set up, as only a single machine is required to maintain it.
- **Cost-Effective:** No additional hardware is required for implementing 1-Tier Architecture, which makes it cost-effective.
- **Easy to Implement:** 1-Tier Architecture can be easily deployed and hence it is mostly used in small projects.

Disadvantages of 1-Tier Architecture

- **Limited to Single User:** Only one person can use the application at a time. It's not designed for multiple users or teamwork.
- **Poor Security:** Since everything is on the same machine, if someone gets access to the system, they can access both the data and the application easily.
- **No Centralized Control:** Data is stored locally, so there's no central database. This makes it hard to manage or back up data across multiple devices.
- **Hard to Share Data:** Sharing data between users is difficult because everything is stored on one computer.

2-Tier Architecture

The 2-tier architecture is similar to a basic client-server model. The application at the client end directly communicates with the database on the server side.



2-Tier Architecture

- On the client side, the user interfaces and application programs are run. The application on the client side establishes a connection with the server side to communicate with the DBMS. For Example: A Library Management System used in schools or small organizations is a classic example of two-tier architecture.
- **Client Layer (Tier 1):** This is the user interface that library staff or users interact with. For example they might use a desktop application to search for books, issue them, or check due dates.
- **Database Layer (Tier 2):** The database server stores all the library records such as book details, user information and transaction logs.
- The client layer sends a request (like searching for a book) to the database layer which processes it and sends back the result. This separation allows the client to focus on the user interface, while the server handles data storage and retrieval.

Advantages of 2-Tier Architecture

- **Easy to Access:** 2-Tier Architecture makes easy access to the database, which makes fast retrieval.
- **Scalable:** We can scale the database easily, by adding clients or upgrading hardware.
- **Low Cost:** 2-Tier Architecture is cheaper than 3-Tier Architecture and Multi-Tier Architecture.
- **Easy Deployment:** 2-Tier Architecture is easier to deploy than 3-Tier Architecture.
- **Simple:** 2-Tier Architecture is easily understandable as well as simple because of only two components.

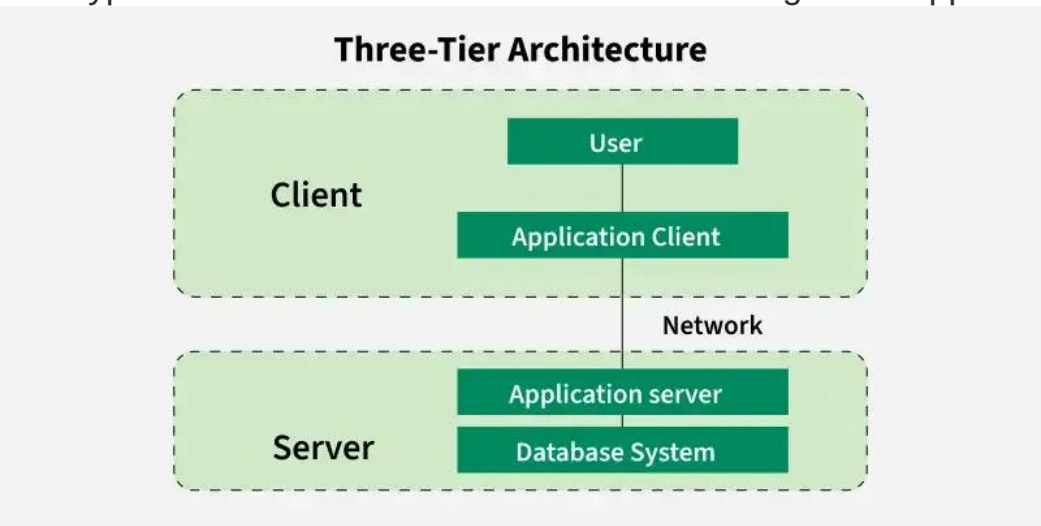
Disadvantages of 2-Tier Architecture

- **Limited Scalability:** As the number of users increases, the system performance can slow down because the server gets overloaded with too many requests.
- **Security Issues:** Clients connect directly to the database, which can make the system more vulnerable to attacks or data leaks.

- **Tight Coupling:** The client and the server are closely linked. If the database changes, the client application often needs to be updated too.
- **Difficult Maintenance:** Managing updates, fixing bugs, or adding features becomes harder when the number of users or systems increases.

3-Tier Architecture

In 3-Tier Architecture, there is another layer between the client and the server. The client does not directly communicate with the server. Instead, it interacts with an application server which further communicates with the database system and then the query processing and transaction management takes place. This intermediate layer acts as a medium for the exchange of partially processed data between the server and the client. This type of architecture is used in the case of large web applications.



DBMS 3-Tier Architecture

Example: E-commerce Store

- **User**: You visit an online store, search for a product and add it to your cart.
- **Processing**: The system checks if the product is in stock, calculates the total price and applies any discounts.
- **Database**: The product details, your cart and order history are stored in the database for future reference.

Advantages of 3-Tier Architecture

- **Enhanced scalability:** Scalability is enhanced due to the distributed deployment of application servers. Now, individual connections need not be made between the client and server.
- **Data Integrity:** 3-Tier Architecture maintains Data Integrity. Since there is a middle layer between the client and the server, data corruption can be avoided/removed.
- **Security:** 3-Tier Architecture Improves Security. This type of model prevents direct interaction of the client with the server thereby reducing access to unauthorized data.

Disadvantages of 3-Tier Architecture

- **More Complex:** 3-Tier Architecture is more complex in comparison to 2-Tier Architecture. Communication Points are also doubled in 3-Tier Architecture.
- **Difficult to Interact:** It becomes difficult for this sort of interaction to take place due to the presence of middle layers.
- **Slower Response Time:** Since the request passes through an extra layer (application server), it may take more time to get a response compared to 2-Tier systems.
- **Higher Cost:** Setting up and maintaining three separate layers (client, server and database) requires more hardware, software and skilled people. This makes it more expensive.

Data Models, Schemas, and Instances

Data Models: Already cover that is DBMS types refer above notes.

Schema

What is Schema?

A **schema** is the **structure or design of a database**.
It tells:

- What tables exist
- Table names
- Columns and their data types
- Relationships and constraints

☞ **Schema is like a blueprint of a building** (design only, not people living in it).

Simple Definition:

Schema defines how the database is organized.

Example of Schema (Simple)

```
STUDENT (RollNo, Name, Class, Marks)
```

This tells:

- Table name: STUDENT
- Columns: RollNo, Name, Class, Marks

No actual student data is stored here—only structure.

Types of Schema

1 Physical Schema

- Describes **how data is stored physically**
- Storage, indexes, file structure
- Used by DBMS internally

Example:

- Data stored in hard disk blocks
- Indexing method (B-tree)

---

2 Logical Schema

- Describes **what data is stored and relationships**
- Tables, attributes, constraints
- Most important for users

Example:

```
STUDENT (RollNo INT, Name VARCHAR, Class VARCHAR)
```

---

3 View Schema

- Describes **user-specific views**
- Partial data shown to users

Example:

```
CREATE VIEW Student_View AS
SELECT Name, Class FROM STUDENT;
```

Schema Example in Oracle

```
CREATE TABLE Student (
    RollNo NUMBER,
    Name VARCHAR2(50),
    Class VARCHAR2(10),
    Marks NUMBER
);
SELECT * FROM Student;
```

RollNo	Name	Class	Marks

Instance

What is Instance?

An **instance** is the **actual data stored in the database at a particular time**.

Simple Definition:

Instance represents the current data in the database.

Example of Instance

RollNo	Name	Class	Marks
1	Ravi	10A	85
2	Anu	10B	90

This data can **change daily** → instance changes.

Types of Instance

1 Initial Instance

- Data at the time of database creation
- Mostly empty

---

2 Current Instance

- Data at present time

---

3 Final Instance

- Data just before database is deleted

Instance Example in Oracle

```
INSERT INTO Student VALUES (1, 'Ravi', '10A', 85);
INSERT INTO Student VALUES (2, 'Anu', '10B', 90);
```

RollNo	Name	Class	Marks
1	Ravi	10A	85
2	Anu	10B	90

3 Difference between Schema and Instance

Feature	Schema	Instance
Meaning	Database structure	Actual data
Nature	Static (rarely changes)	Dynamic (changes frequently)
Time	Defined once	Changes with time
Example	Table definition	Table records
Oracle Example	`CREATE TABLE`	`INSERT INTO`
Comparison	Blueprint	Real data
Stored	In data dictionary	In database tables

PL/ SQL Data Types

Numeric Data Types and Subtypes in PL/SQL

Numeric data types store numbers, both integers and real numbers, and allow developers to perform arithmetic operations. The main numeric types include:

- **NUMBER**: A highly flexible type that can store fixed-point or floating-point numbers. It has precision and scale parameters. For example, **NUMBER(5, 2)** can store up to 5 digits, with 2 of them after the decimal point.
- **BINARY_INTEGER/PLS_INTEGER:** These types are used for signed integers and are often faster than the generic NUMBER type because they use machine-dependent formats.
- **FLOAT**: It is a subtype of NUMBER designed for storing floating-point numbers. You can specify an optional precision such as **FLOAT(10)** which allows for storing a number with up to 10 digits of precision

Character Data Types and Subtypes in PL/SQL

Character data types are designed to store any text, numbers or symbols in the form of alphanumeric. They are used specifically for string manipulation and are a vital part of PL/SQL.

- **CHAR**: It Handles variable-length binary data and fixed-length character strings. If the string is less than the defined length, then the rest is filled up with spaces. For instance, CHAR(10) would store string as CHAR data type that has a length of 10 characters regardless of the actual string length.
- **VARCHAR2**: To store character strings of varying lengths. **VARCHAR2** is slightly different because it only allocates the required amount of space required to store the string. For instance, **VARCHAR2(10)** data type can accommodate a string with as many as 10 characters.
- **LONG**: It Can store variable-length character strings of up to 2 gigabytes. However, it is deprecated and it should be replaced with **CLOB** to 2 GB. However, it is deprecated and should be avoided in favor of CLOB.

PL/SQL Datetime and Interval Types

Datetime data types contain date and time and interval types contain the difference between two datetime values. PL/SQL provides the following datetime and interval types:

- **DATE:** It stores **date** and **time** values. These are the year, month, day, hour, minute, and second as a part of the Date type.
- **TIMESTAMP**: It is an extension of the DATE data type with the added feature of fractional seconds.
- **TIMESTAMP WITH TIME ZONE**: Saves a TIMESTAMP, but with information about the time zone in which it has been set.
- **TIMESTAMP WITH LOCAL TIME ZONE**: Standalone function that converts the TIMESTAMP to the time zone of the current database session.
- **INTERVAL YEAR TO MONTH:** Saves the amount of time measured in years and months.
- **INTERVAL DAY TO SECOND**: Saves as the time duration in terms of days, hours, minutes, and seconds.

CustomerID	FirstName	LastName	Country	Age	Phone
1	Luca	Bianchi	Italy	23	xxxxxxxxxx
2	Aiko	Tanaka	Japan	21	xxxxxxxxxx
3	Carlos	Gomez	Spain	24	xxxxxxxxxx
4	Sofia	Müller	Germany	22	xxxxxxxxxx
5	Ethan	Johnson	USA	25	xxxxxxxxxx

Q. For above table write Oracle Query

Data Independence in DBMS

Definition

Data Independence in DBMS means the **ability to change the database structure at one level without affecting the next higher level**.
It allows changes in data storage or schema **without changing application programs**.

Types of Data Independence

There are **two types** of data independence:

---

1 Physical Data Independence

Meaning

Ability to change the **physical storage of data** without affecting the **logical schema or application programs**.

What can be changed?

- File organization
- Indexing
- Storage location
- Compression techniques

Example

- Changing data storage from **HDD to SSD**
- Adding an index to a table

```
CREATE INDEX idx_marks ON Student(Marks);
```

✓ This change **does not affect** how users query the data.

---

2 Logical Data Independence

Meaning

Ability to change the **logical schema** without affecting the **external schema or application programs**.

What can be changed?

- Adding/removing columns
- Splitting tables
- Changing relationships

Example

Adding a new column to a table:

```
ALTER TABLE Student ADD Address VARCHAR2(50);
```

✓ Existing programs still work without modification.

Types of Database Users

1 Naive (End) Users

- Use predefined application programs
- Do not write SQL queries

Examples:

- Bank customer using ATM
- Online shopping user

---

2 Application Programmers

- Develop database applications
- Write SQL and PL/SQL code

Examples:

- Software developers
- Web application developers

3 Sophisticated Users

- Use SQL queries directly
- Perform complex data analysis

Examples:

- Data analysts
- Engineers, researchers

---

4 Specialized Users

- Write special-purpose database programs

Examples:

- CAD/CAM users
- Scientific database users

Database Administrator (DBA)

A **Database Administrator (DBA)** is the person **responsible for managing and controlling the database system**.

Responsibilities of DBA

- Database installation and configuration
- Creating users and granting privileges
- Database security and authorization
- Backup and recovery
- Performance monitoring and tuning
- Data integrity and availability

ENTITY–RELATIONSHIP (ER) MODEL – DETAILED NOTES

Introduction to ER Model

The **Entity–Relationship (ER) Model** is a **conceptual data model** used in **database design**.
It visually represents the **structure of a database** before converting it into tables.

Why ER Model is used

- Easy to understand real-world data
- Helps in **conceptual design**
- Acts as a **blueprint** for database creation
- Reduces redundancy and inconsistency
- ER diagrams represent the E-R model in a database, making them easy to convert into relations (tables).
- These diagrams serve the purpose of real-world modelling of objects which makes them intently useful.

- Unlike technical schemas, ER diagrams require no technical knowledge of the underlying DBMS used.
- They visually model data and its relationships, making complex systems easier to understand.

Developed by

- **Peter Chen (1976)**

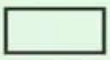
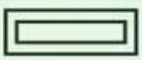
Basic Components of ER Model

The ER model consists of:

1. **Entity**
2. **Attributes**
3. **Relationship**
4. **Constraints**

- **Entity:** An object that is stored as data. E.g: *Student*, *Course*, or *Company*.
- **Attribute:** Properties that describe an entity. E.g: *StudentID*, *CourseName*, or *EmployeeEmail*.
- **Relationship:** A connection between entities. E.g: *Student* enrolls in a *Course*.

Notations for ER diagram

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

Entity

An Entity represents a real-world object, concept or thing about which data is stored in a database.

Entity

An **entity** is a **real-world object** that is distinguishable from others.

📌 Examples:

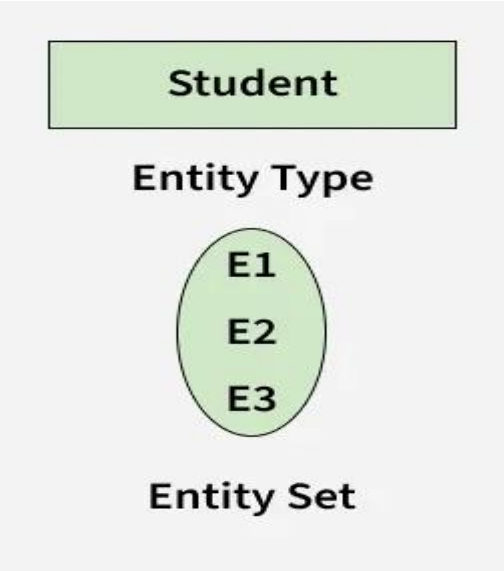
- Student
- Employee
- Department
- Book

Entity Set

A **collection of similar entities**.

📌 Example:

- All students in a college form the **Student entity set**



.

Types of Entity

1. Strong Entity

- Has a **primary key**
- Exists independently

📌 Example:

Student (Student_ID, Name, Age)

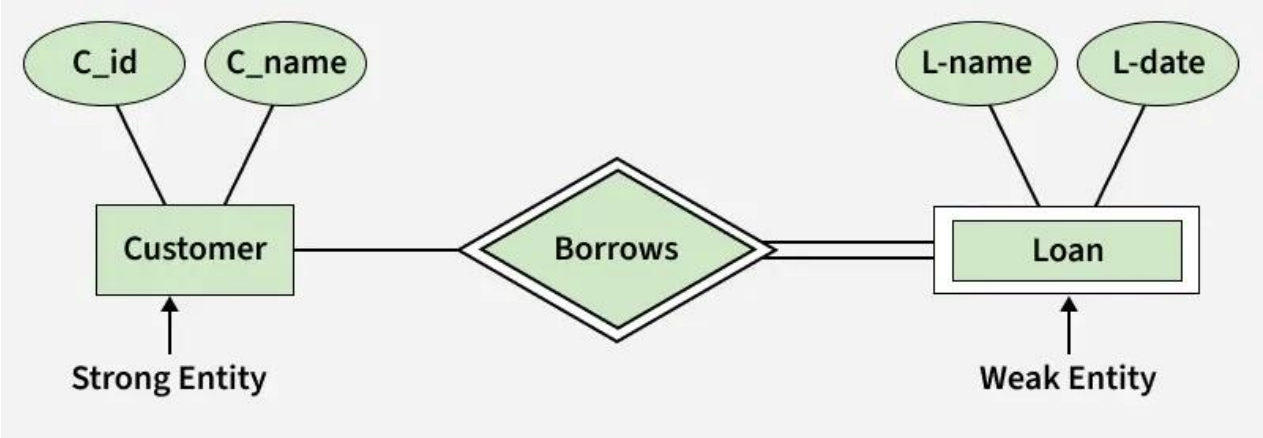
2. Weak Entity

- Does **not have a primary key**
- Depends on a strong entity
- Uses **partial key**

📌 Example:

Dependent (Dependent_Name, Age)

Dependent depends on Employee.



Attributes

Attributes describe the **properties of an entity or relationship**.

Types of Attributes

1. Simple Attribute

- Cannot be divided further

📌 Example:

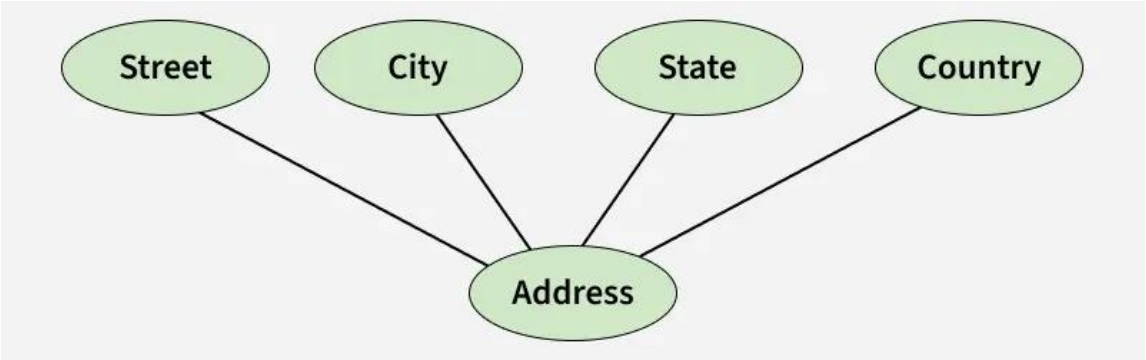
- Age
- Salary

2. Composite Attribute

- Can be divided into sub-parts
- An attribute composed of many other attributes is called a composite attribute. For example, the Address attribute of the student Entity type consists of Street, City, State, and Country. In ER diagram

📌 Example:

```
Name → First_Name, Middle_Name, Last_Name
```



3. Single-valued Attribute

- Only one value per entity

📌 Example:

- Roll_Number

4. Multi-valued Attribute

- Multiple values allowed

📌 Example:

- Phone_Number
- Email_IDs

(Shown using **double oval** in ER diagram)

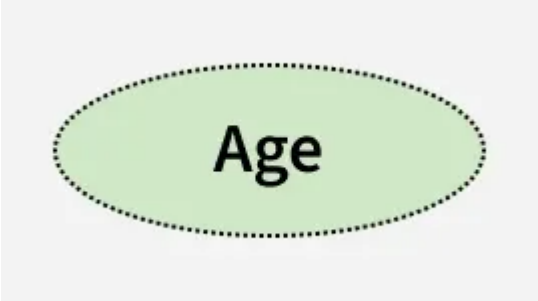
5. Derived Attribute

- Calculated from other attributes

📌 Example:

- Age (derived from Date_of_Birth)

(Shown using **dashed oval**)



6. Key Attribute

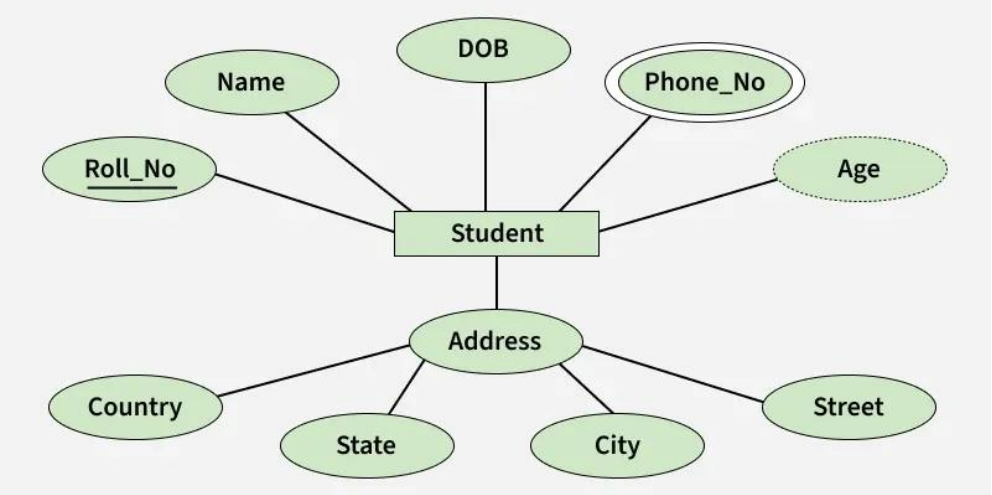
- Uniquely identifies an entity

📌 Example:

- Student_ID

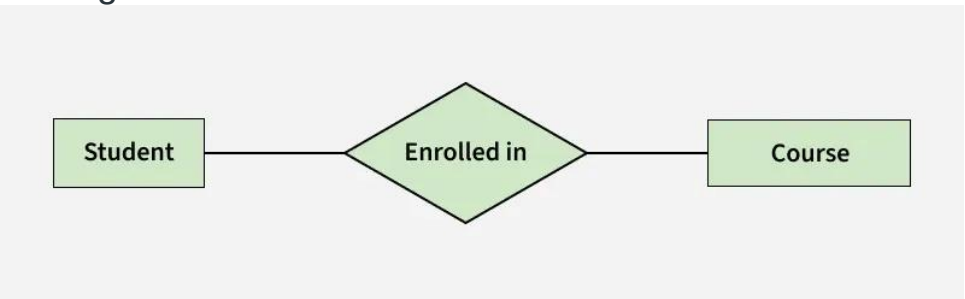
(Shown by **underlining**)

The Complete Entity Type Student with its Attributes can be represented as:



Relationship Type and Relationship Set

- A Relationship Type represents the association between entity types.
- For example, ‘Enrolled in’ is a relationship type that exists between entity type Student and Course.
- In ER diagram, the relationship type is represented by a diamond and connecting the entities with lines.



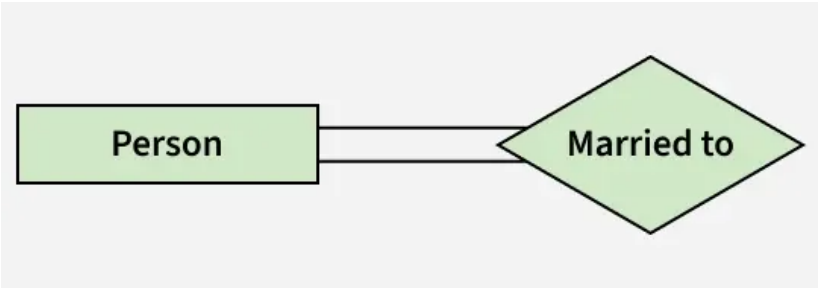
Entity-Relationship Set

Degree of a Relationship Set

The number of different entity sets participating in a relationship set is called the degree of a relationship set.

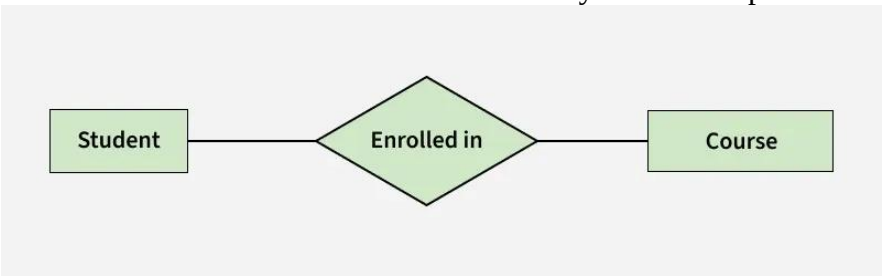
1. Unary/Recursive Relationship: When there is only ONE entity set participating in a relation, the relationship is called a unary relationship. For example, one person is married to only one person.

Unary Relationship



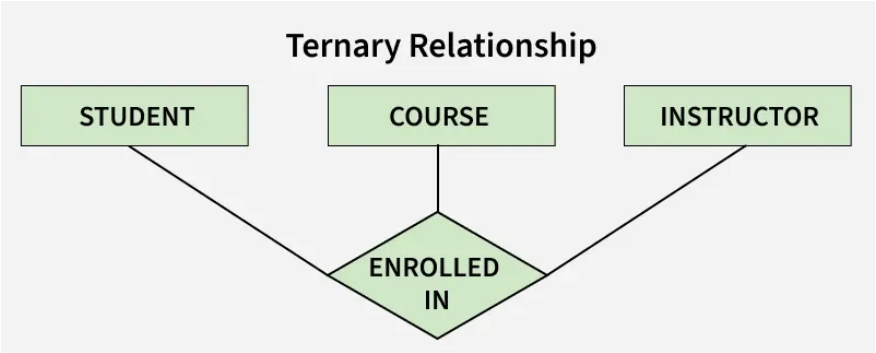
2. Binary Relationship: When there are TWO entities set participating in a relationship, the relationship is called a binary relationship. For example, a Student is enrolled in a Course.

Binary Relationship

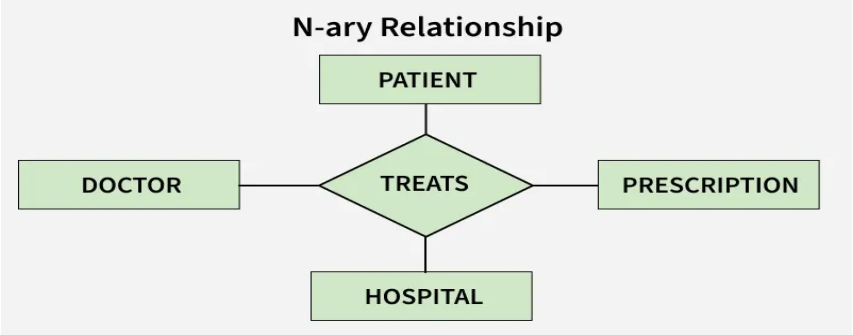


3. Ternary Relationship: When there are three entity sets participating in a relationship, the relationship is called a ternary relationship.

Ternary Relationship



5. **N-ary Relationship:** When there are n entities set participating in a relationship, the relationship is called an n-ary relationship.



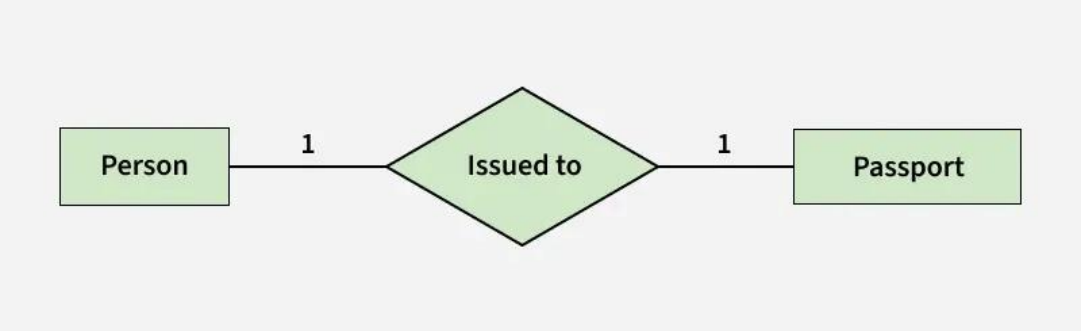
N-ary Relationship

Cardinality in ER Model

The maximum number of times an entity of an entity set participates in a relationship set is known as cardinality.
Cardinality can be of different types:

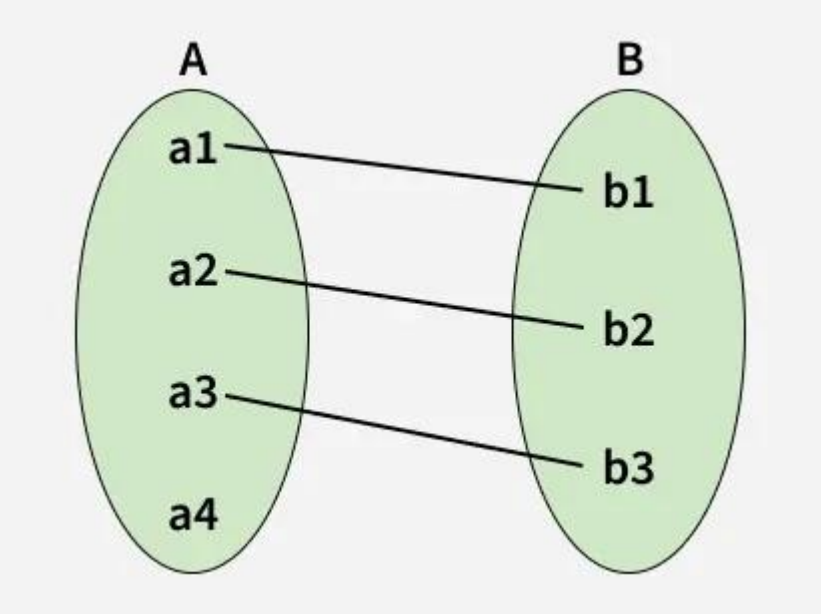
1. One-to-One

When each entity in each entity set can take part only once in the relationship



one to one cardinality

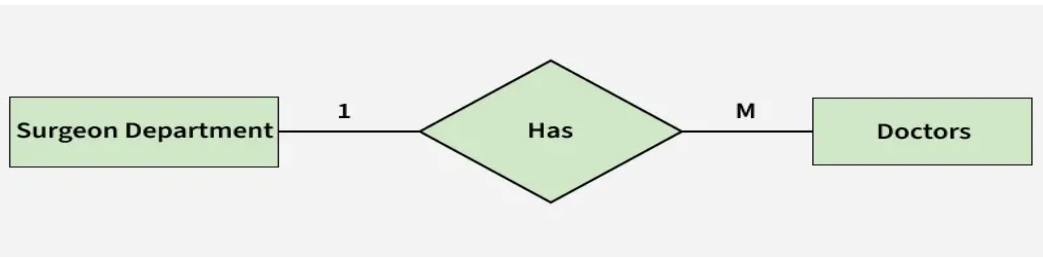
Using Sets, it can be represented as:



Set Representation of One-to-One

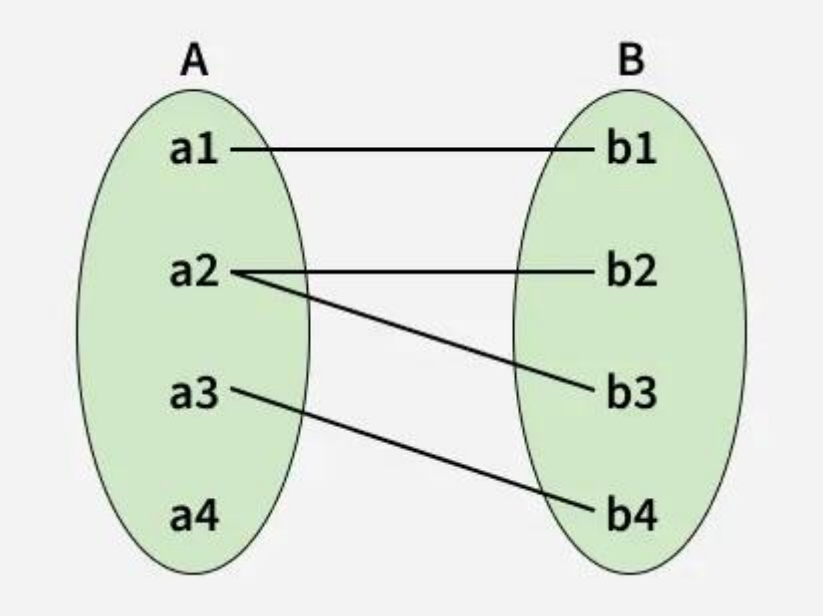
2. One-to-Many

In a one-to-many relationship, one entity can be associated with multiple entities.



one to many cardinality

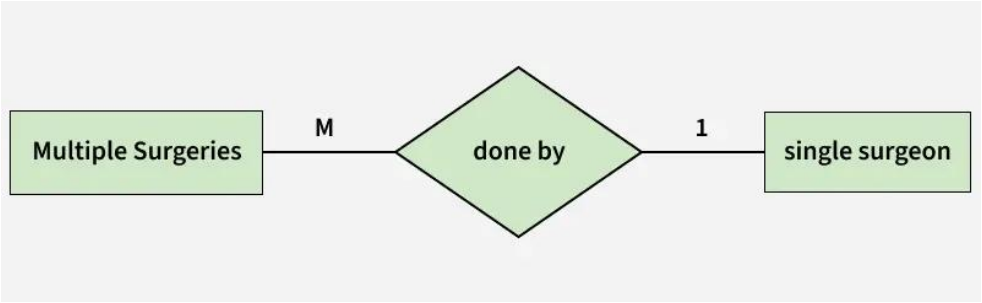
Using sets, one-to-many cardinality can be represented as:



Set Representation of One-to-Many

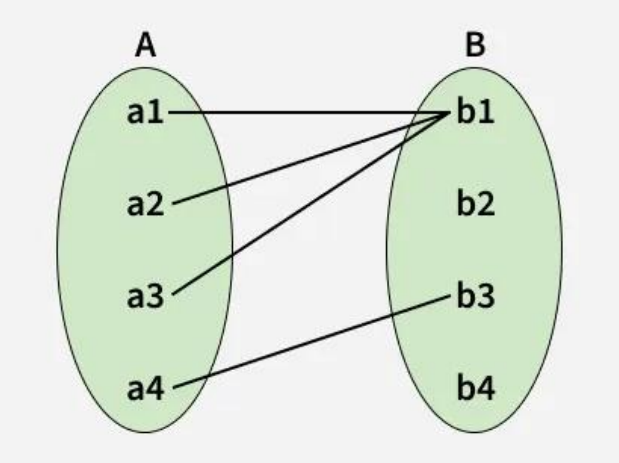
3. Many-to-One

When entities in one entity set can take part only once in the relationship set and entities in other entity sets can take part more than once in the relationship set, cardinality is many to one.



many to one cardinality

Using Sets, it can be represented as:

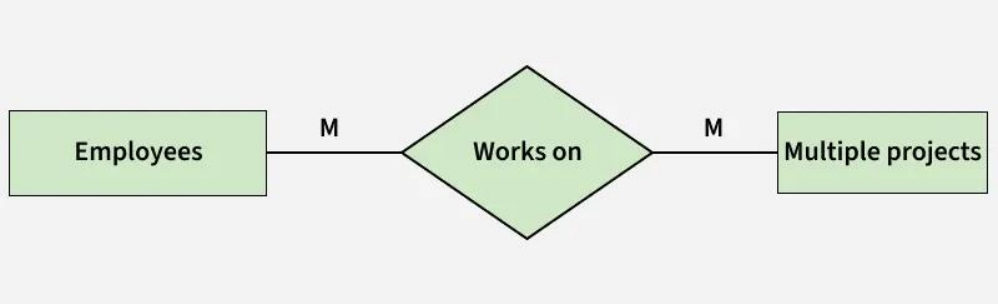


Set Representation of Many-to-One

In this case, each student is taking only 1 course but 1 course has been taken by many students.

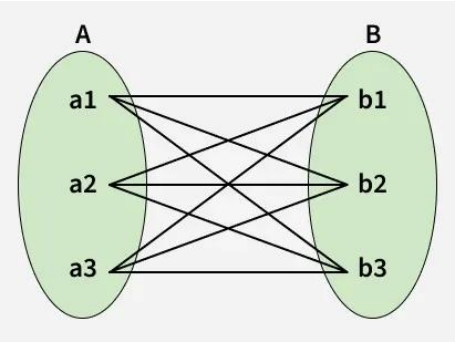
4. Many-to-Many

When entities in all entity sets can take part more than once in the relationship cardinality is many to many.



many to many cardinality

Using Sets, it can be represented as:



Many-to-Many Set Representation

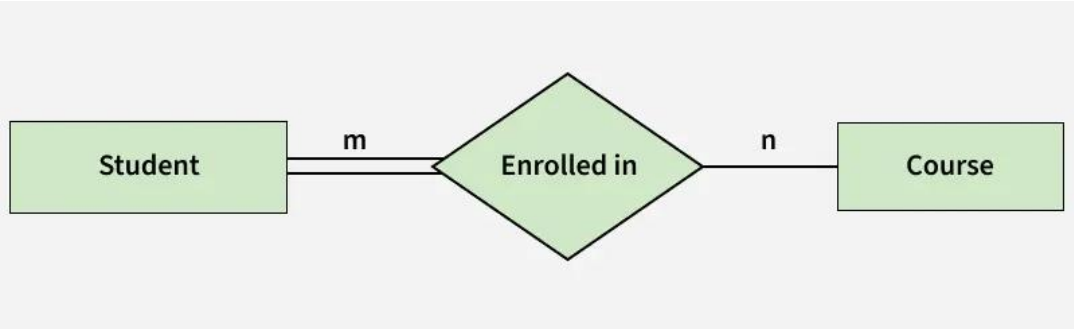
In this example, student A1 is enrolled in B1,B2 and B3, and course B3 is taken by A1, A2, and A3. Therefore, this represents a many-to-many relationship.

Participation Constraint

Participation Constraint is applied to the entity participating in the relationship set.

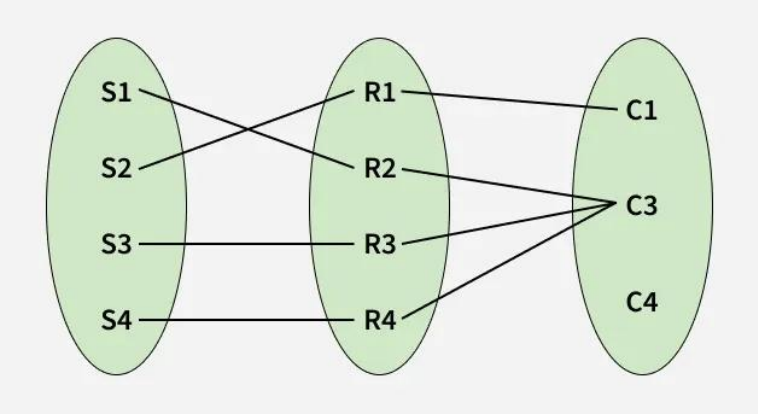
1. **Total Participation:** Each entity in the entity set must participate in the relationship. If each student must enroll in a course, the participation of students will be total. Total participation is shown by a double line in the ER diagram.
2. **Partial Participation:** The entity in the entity set may or may NOT participate in the relationship. If some courses are not enrolled by any of the students, the participation in the course will be partial.

The diagram depicts the 'Enrolled in' relationship set with Student Entity set having total participation and Course Entity set having partial participation.



Total Participation and Partial Participation

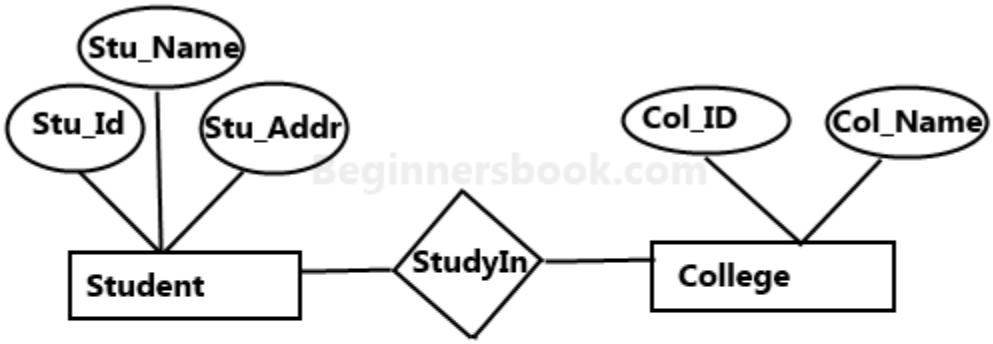
Using Set, it can be represented as,



How to Draw an ER Diagram

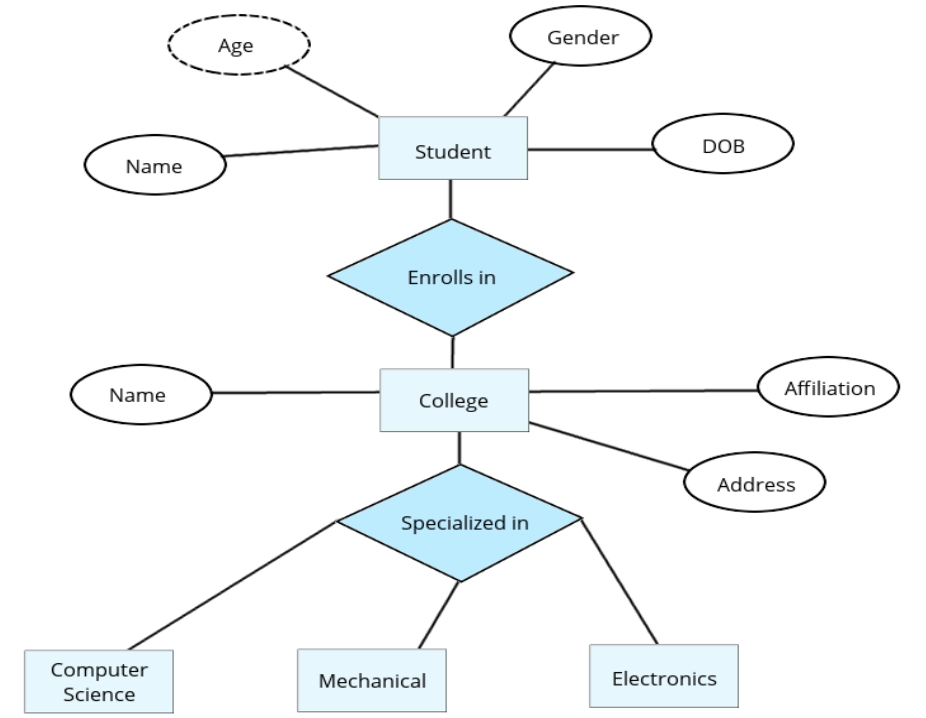
1. **Identify Entities:** The very first step is to identify all the Entities. Represent these entities in a Rectangle and label them accordingly.
2. **Identify Relationships**: The next step is to identify the relationship between them and represent them accordingly using the Diamond shape. Ensure that relationships are not directly connected to each other.

3. **Add Attributes**: Attach attributes to the entities by using ovals. Each entity can have multiple attributes (such as name, age, etc.), which are connected to the respective entity.
4. **Define Primary Keys:** Assign primary keys to each entity. These are unique identifiers that help distinguish each instance of the entity. Represent them with underlined attributes.
5. **Remove Redundancies**: Review the diagram and eliminate unnecessary or repetitive entities and relationships.
6. **Review for Clarity**: Review the diagram make sure it is clear and effectively conveys the relationships between the entities.

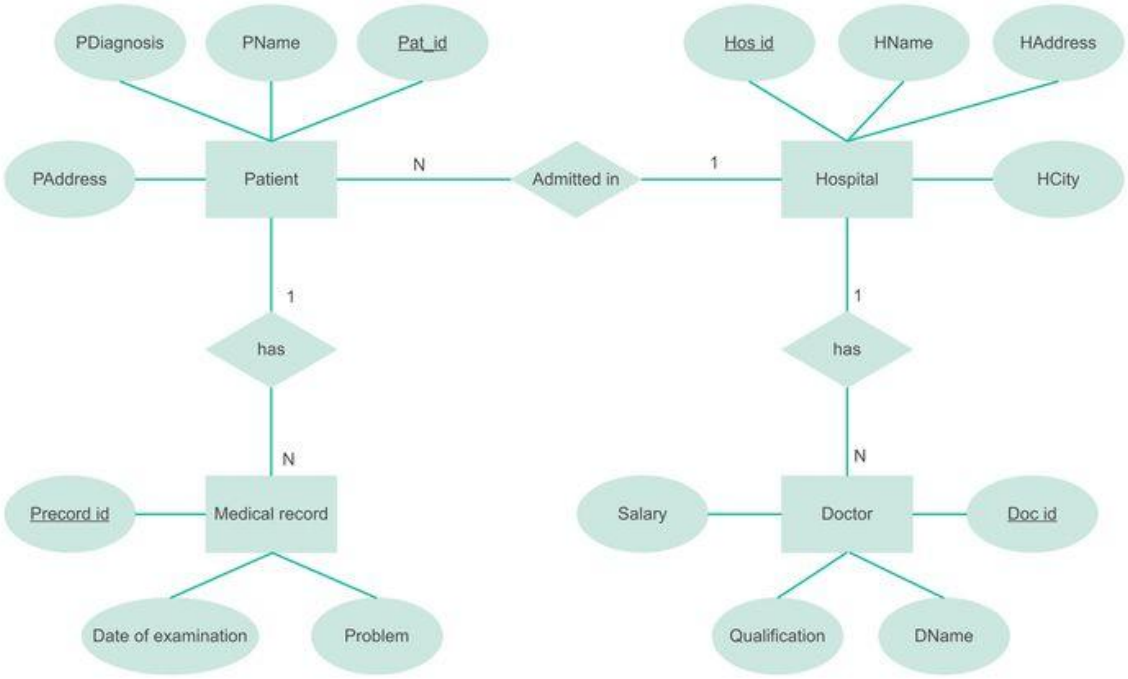


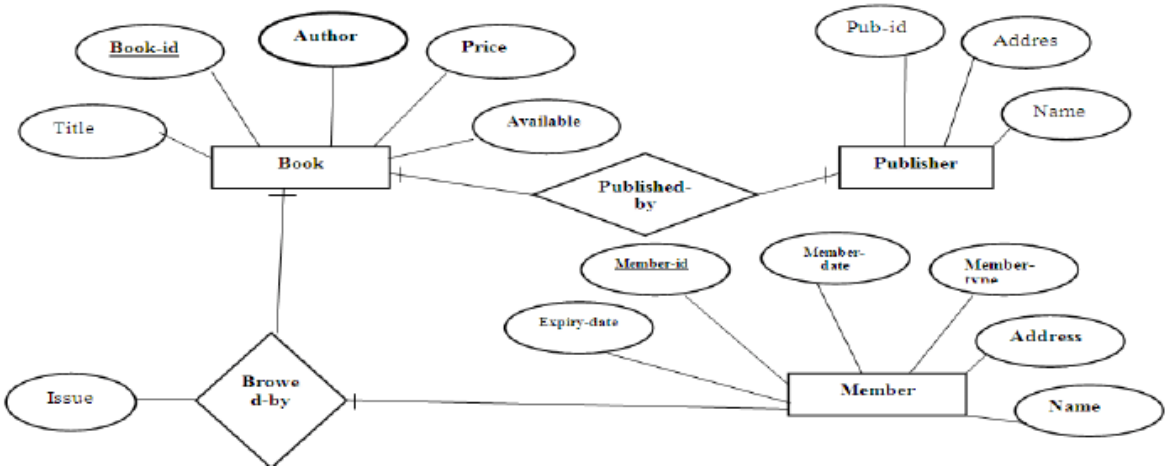
Sample E-R Diagram

ER diagram for college simple example

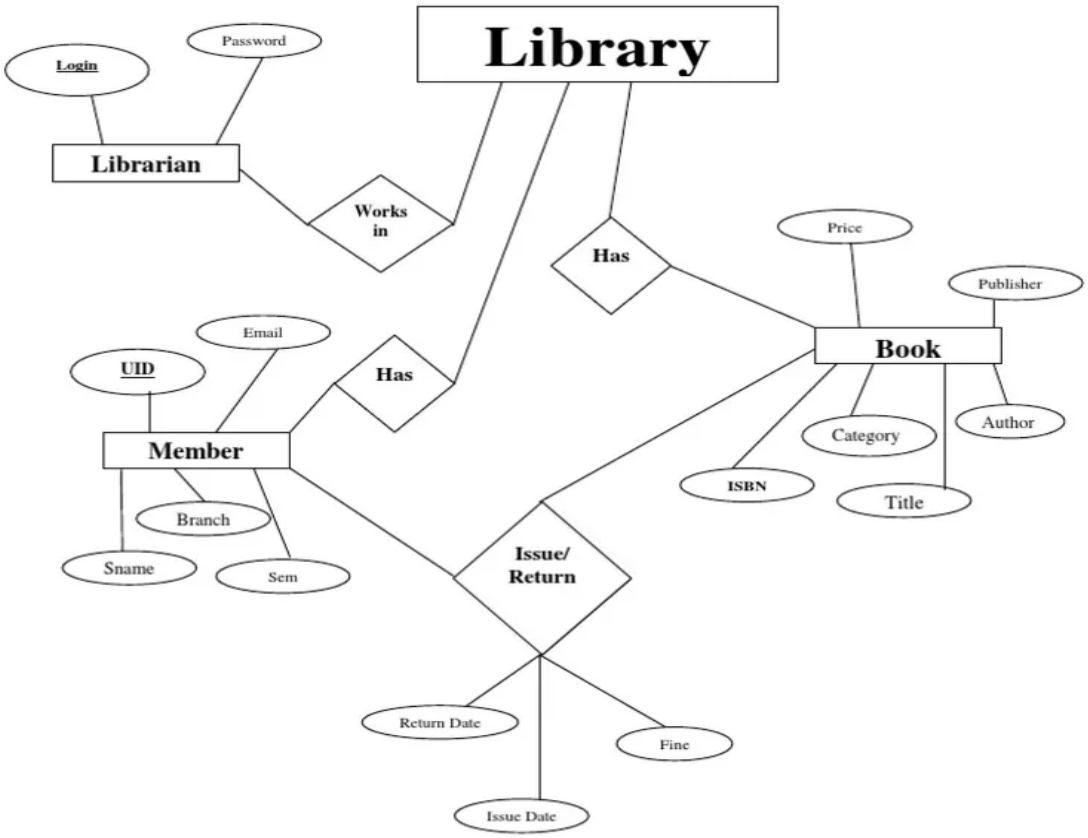

Age
Gender
Name
Student
DOB
Enrolls in
Name
College
Affiliation
Address
Specialized in
Computer Science
Mechanical
Electronics

ER Diagram of Hospital Management System





ER Diagram: Library Management System



- Construct an E-R diagram for a car-insurance company whose customers own one or more cars each. Each car has associated with it zero to any number of recorded accidents.

Step 1 : Identify the entity sets

- From the given question the entity sets identified are
  1. PERSON
  2. CAR
  3. ACCIDENT

Step 2 : Identify the relevant attributes

- The relevant attributes of PERSON entity set are
  - Person_Id
  - Person_Name
  - Address
- The relevant attributes of CAR entity set are
  - Engine_No
  - Model
  - Year
- The relevant attributes of ACCIDENT entity set are
  - Report_No
  - Location
  - Acc_Date

Step 3 : Identify the prime attributes

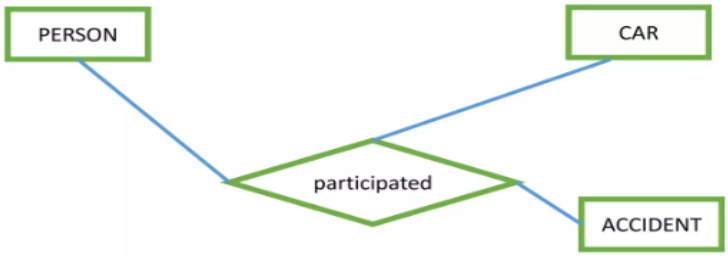
- The prime attribute of PERSON entity set is
  - Person_Id
- The prime attribute of CAR entity set is
  - Engine_No
- The prime attribute of ACCIDENT entity set is
  - Report_No

Step 4 : Identify the relationships

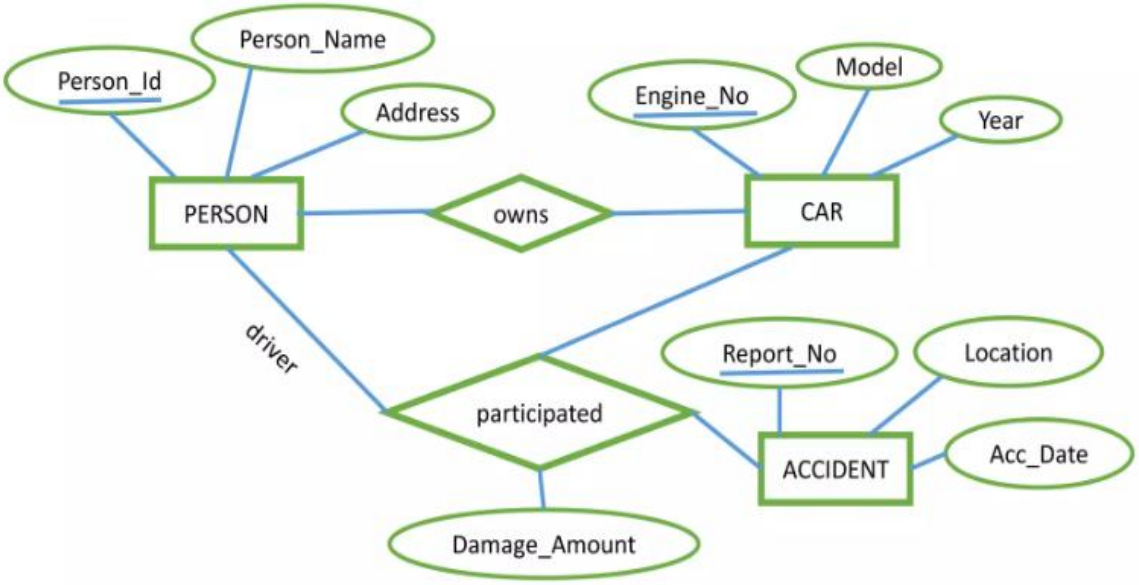
- Customers own one or more cars each.

Step 4 : Identify the relationships

- Each car is associated with it zero to any number of recorded accidents.



Step 5 : Complete E R Diagram



- Design an E-R diagram for keeping track of the exploits of your favorite sports team. You should store the matches played, the scores in each match, the players in each match and individual player statistics for each match. Summary statistics should be modeled as derived attributes.

Step 1 : Identify the entity sets

- From the given question the entity sets identified are
  1. MATCH
  2. PLAYER

Step 2 : Identify the relevant attributes

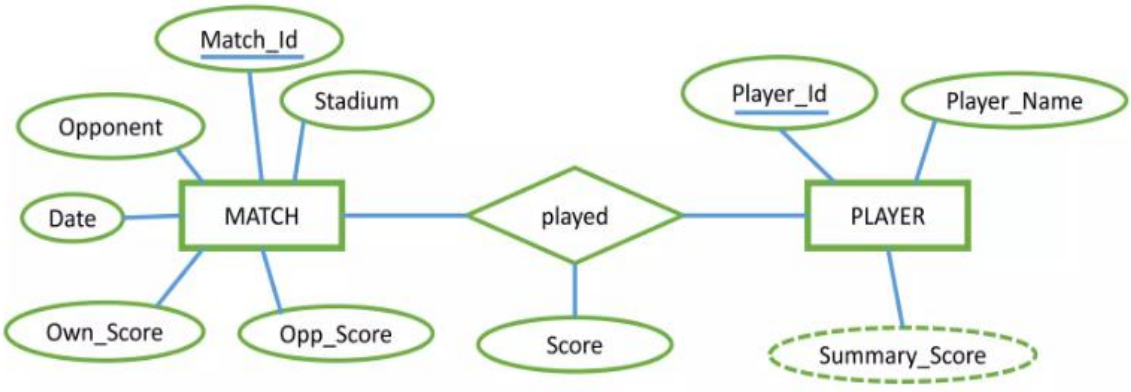
- The relevant attributes of MATCH entity set are
  - Match_Id
  - Stadium
  - Date
  - Opponent
  - Own_Score
  - Opp_Score
- The relevant attributes of PLAYER entity set are
  - Player_Id
  - Player_Name
  - Summary_Score

Step 3 : Identify the prime attributes

- The prime attribute of MATCH entity set is
  - Match_Id
- The prime attribute of PLAYER entity set is
  - Player_Id

Step 4 : Identify the relationships

Step 5 : Complete E R Diagram



ER Diagram Practice Questions

Question1: Design an ER diagram for a university that has students, professors, and courses. Students enroll in courses, and professors teach courses. Include appropriate attributes and relationships with cardinality. (University System)

Question2: "A hospital has multiple departments. Each department has doctors. A doctor can work in more than one department. Patients visit doctors, and each visit is recorded." Draw an ER diagram to represent the scenario with all entities, relationships, and cardinalities. (Hospital Management System)

Question3: Create an ER diagram showing books, members, borrow/return transactions, and fine records. Books may have multiple authors. A member can borrow many books but one at a time. (Library Management System)

Question4: Model customers, orders, products, payment, and delivery as entities. Show relationships such as a customer placing orders, orders containing products, and products having categories. (Online shopping system)

Question5: Design an ER diagram for an airline system where passengers book flights. Flights are scheduled by different airlines and there multiple crew members. (Airline Reservation System)

Question 1

Concepts
Entities: Students, Professors, Courses Attributes: StudentID, StudentName, ProfessorID, ProfessorName, CourseID, CourseName Relationships: Enrolls, Teaches

Explanation
In this university system, students enroll in courses, and professors teach those courses. The cardinality is many-to-many between students and courses, and one-to-many between professors and courses.

Solution

1. **Entities**:
    - **Student**: StudentID (PK), StudentName
    - **Professor**: ProfessorID (PK), ProfessorName
    - **Course**: CourseID (PK), CourseName
2. **Relationships**:
    - **Enrolls**: between Student and Course (Many-to-Many)
    - **Teaches**: between Professor and Course (One-to-Many)

Question 2

Concepts
Entities: Departments, Doctors, Patients, Visits Attributes: DepartmentID, DoctorID, PatientID, VisitID Relationships: WorksIn, Visits

Explanation
In the hospital system, departments have doctors, and patients visit doctors. A doctor can work in multiple departments, and each visit is recorded.

Solution
1. **Entities**:
    - **Department**: DepartmentID (PK)
    - **Doctor**: DoctorID (PK)
    - **Patient**: PatientID (PK)
    - **Visit**: VisitID (PK)
2. **Relationships**:
    - **WorksIn**: between Doctor and Department (Many-to-Many)
    - **Visits**: between Patient and Doctor (Many-to-Many)

Question 3
Concepts
Entities: Books, Members, Transactions, Fines Attributes: BookID, MemberID, TransactionID, FineAmount Relationships: Borrows, Returns

Explanation
In the library system, members can borrow books, and each transaction is recorded. A member can borrow many books but only one at a time.

Solution
1. **Entities**:
    - **Book**: BookID (PK)
    - **Member**: MemberID (PK)
    - **Transaction**: TransactionID (PK)
    - **Fine**: FineID (PK), FineAmount
2. **Relationships**:
    - **Borrows**: between Member and Book (One-to-Many)
    - **Records**: between Transaction and Book (One-to-Many)

Question 4
Concepts
Entities: Customers, Orders, Products, Payments, Deliveries Attributes: CustomerID, OrderID, ProductID, PaymentID, DeliveryID Relationships: Places, Contains, Has

Explanation
In the online shopping system, customers place orders, which contain products. Each order has a payment and delivery associated with it.

Solution

1. **Entities**:
    - **Customer**: CustomerID (PK)
    - **Order**: OrderID (PK)
    - **Product**: ProductID (PK)
    - **Payment**: PaymentID (PK)
    - **Delivery**: DeliveryID (PK)
2. **Relationships**:
    - **Places**: between Customer and Order (One-to-Many)
    - **Contains**: between Order and Product (Many-to-Many)
    - **Has**: between Order and Payment (One-to-One)
    - **Has**: between Order and Delivery (One-to-One)

Question 5

Concepts

Entities: Passengers, Flights, Airlines, Crew Members Attributes: PassengerID, FlightID, AirlineID, CrewID Relationships: Books, Schedules

Explanation

In the airline system, passengers book flights, which are scheduled by different airlines and have multiple crew members.

Solution

1. **Entities**:
    - **Passenger**: PassengerID (PK)
    - **Flight**: FlightID (PK)
    - **Airline**: AirlineID (PK)
    - **Crew**: CrewID (PK)
2. **Relationships**:
    - **Books**: between Passenger and Flight (Many-to-Many)
    - **Schedules**: between Airline and Flight (One-to-Many)
    - **Has**: between Flight and Crew (Many-to-Many)

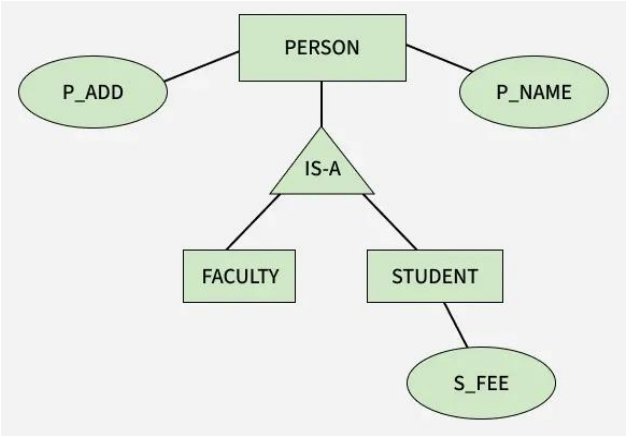
EER Extended ER diagram features

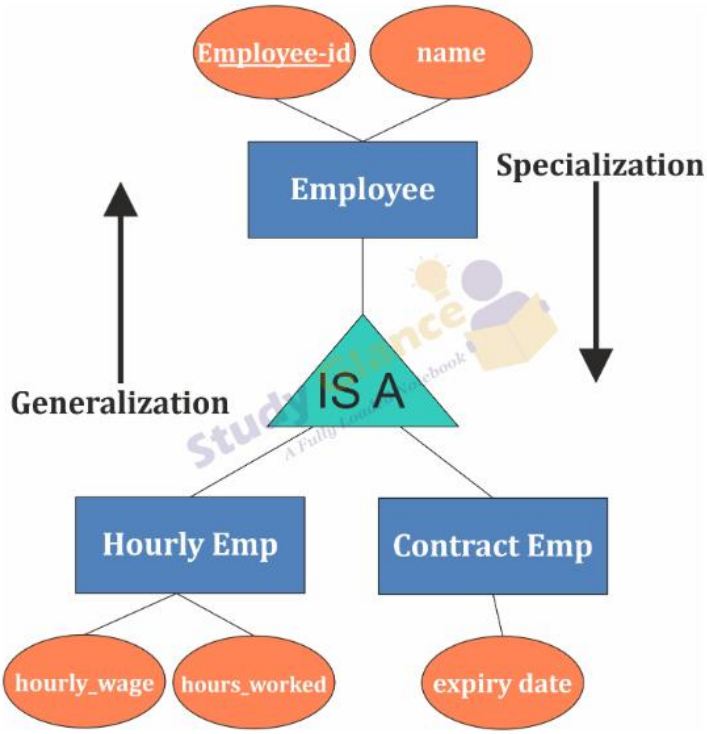
Generalization, Specialization and Aggregation in ER Model

- As the complexity of data increased in the late 1980s, it became more and more difficult to use the traditional ER Model for database modelling. Hence some improvements or enhancements were made to the existing ER Model to make it able to handle the complex applications better.
- Hence, as part of the **Extended ER Model**, along with other improvements, three new concepts were added to the existing ER Model:
  1. **Generalization**
  2. **Specialization**
  3. **Aggregation**

Generalization

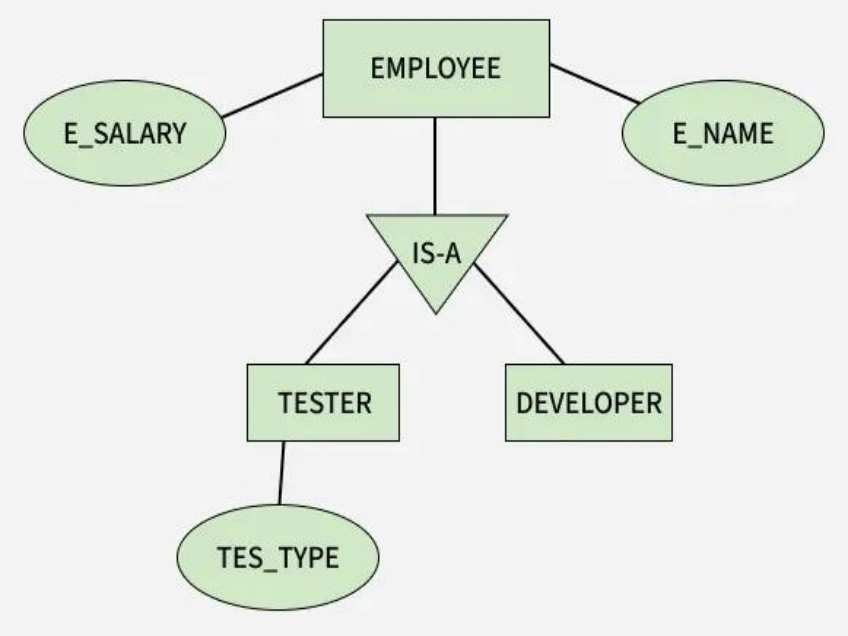
- Process of extracting common properties from a set of entities and creating a generalized entity from it.
- It is a bottom-up approach in which two or more entities can be generalized to a higher-level entity if they have some attributes in common.
- ***Example*** *STUDENT and FACULTY can be generalized to a higher-level entity called PERSON as shown in diagram below. In this case, common attributes like P_NAME and P_ADD become part of a higher entity (PERSON) and specialized attributes like S_FEE become part of a specialized entity (STUDENT).*

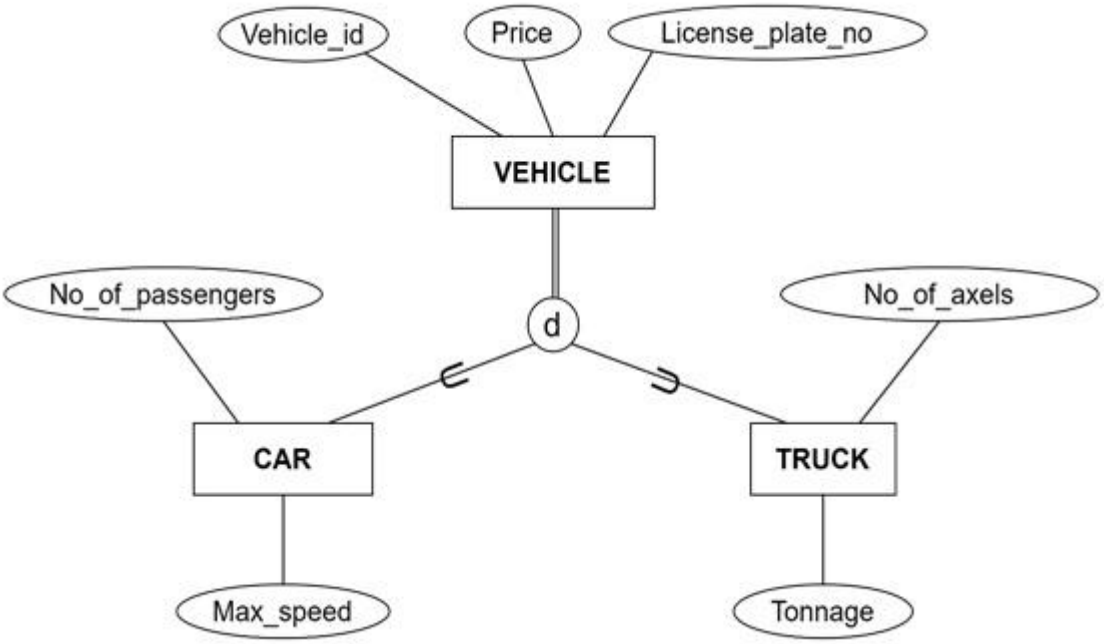


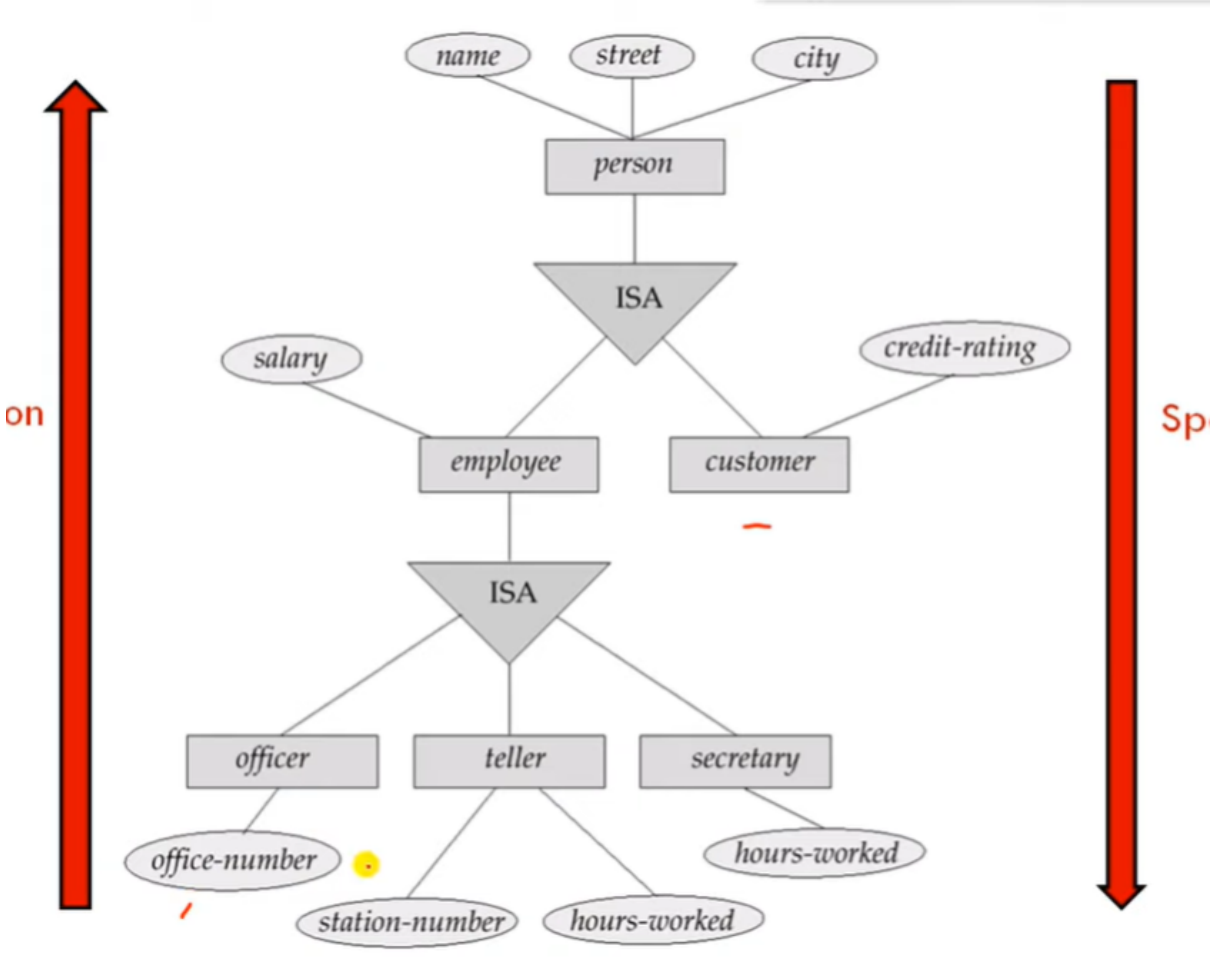


Specialization

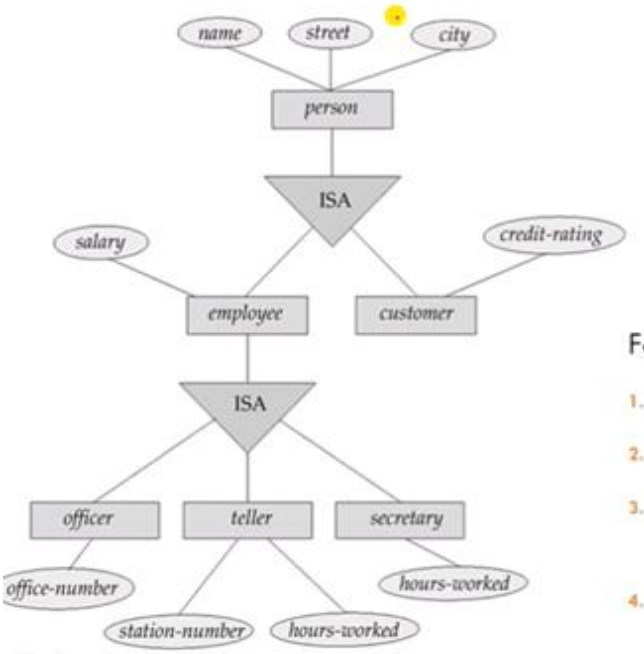
In specialization, an entity is divided into sub-entities based on its characteristics. It is a top-down approach where the higher-level entity is specialized into two or more lower-level entities.




Vehicle_id
Price
License_plate_no
VEHICLE
No_of_passengers
d
No_of_axels
CAR
TRUCK
Max_speed
Tonnage


name
street
city
person
ISA
salary
credit-rating
employee
customer
ISA
officer
teller
secretary
office-number
hours-worked
station-number
hours-worked

How Schema or Tables can be formed?



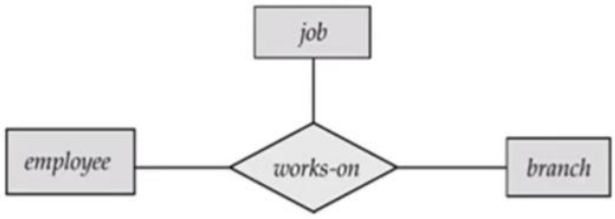
Four tables can be formed:

1. **customer** (name, street, city, credit_rating)
2. **officer** (name, street, city, salary, office_number)
3. **teller** (name, street, city, salary, station_number, hours_worked)
4. **secretary** (name, street, city, salary, hours_wo

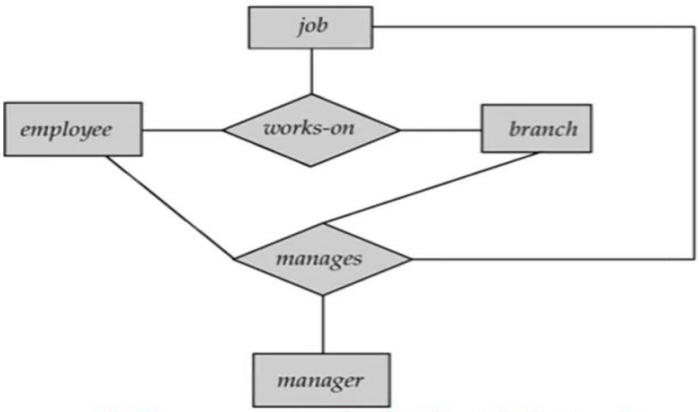
Aggregation

- **Aggregation** is used when we need to express a **relationship among relationships**
- **Aggregation** is an **abstraction** through which **relationships are treated as higher level entities**
- **Aggregation** is a process when a **relationship between two entities is considered as a single entity** and again this single entity has a **relationship** with **another entity**

- Basic E-R model can't represent relationships involving other relationships
- Consider a ternary relationship **works_on** between **Employee, Branch and Job.**
  - Am **employee *works on*** a particular **job** at a particular **branch**



- Suppose we want to assign a **manager** for jobs performed by an employee at a branch (i.e. want to assign managers to each employee, job, branch combination)
  - Need a separate manager entity-set
  - Relationship between each manager, employee, branch, and job entity



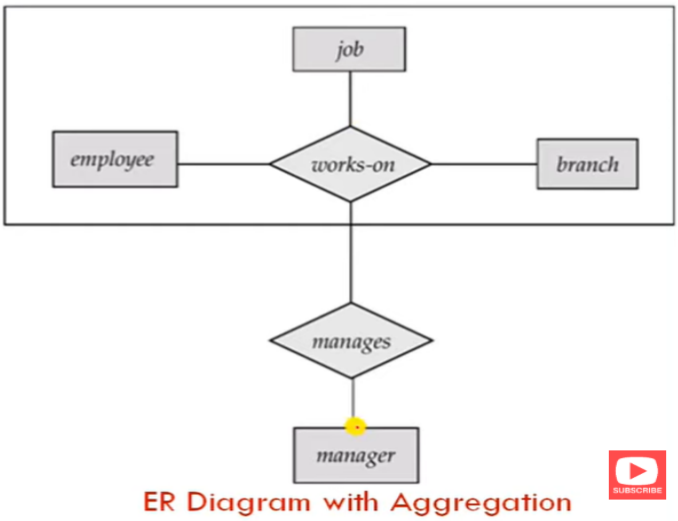
ER Diagram with Redundant Relationship

❑ Relationship sets ***works-on*** and ***manages*** represent overlapping (redundant) information

- ➢ Every *manages* relationship corresponds to a *works-on* relationship
- ➢ However, some *works-on* relationships may not correspond to any *manages* relationships
  - So we can't discard the *works-on* relationship

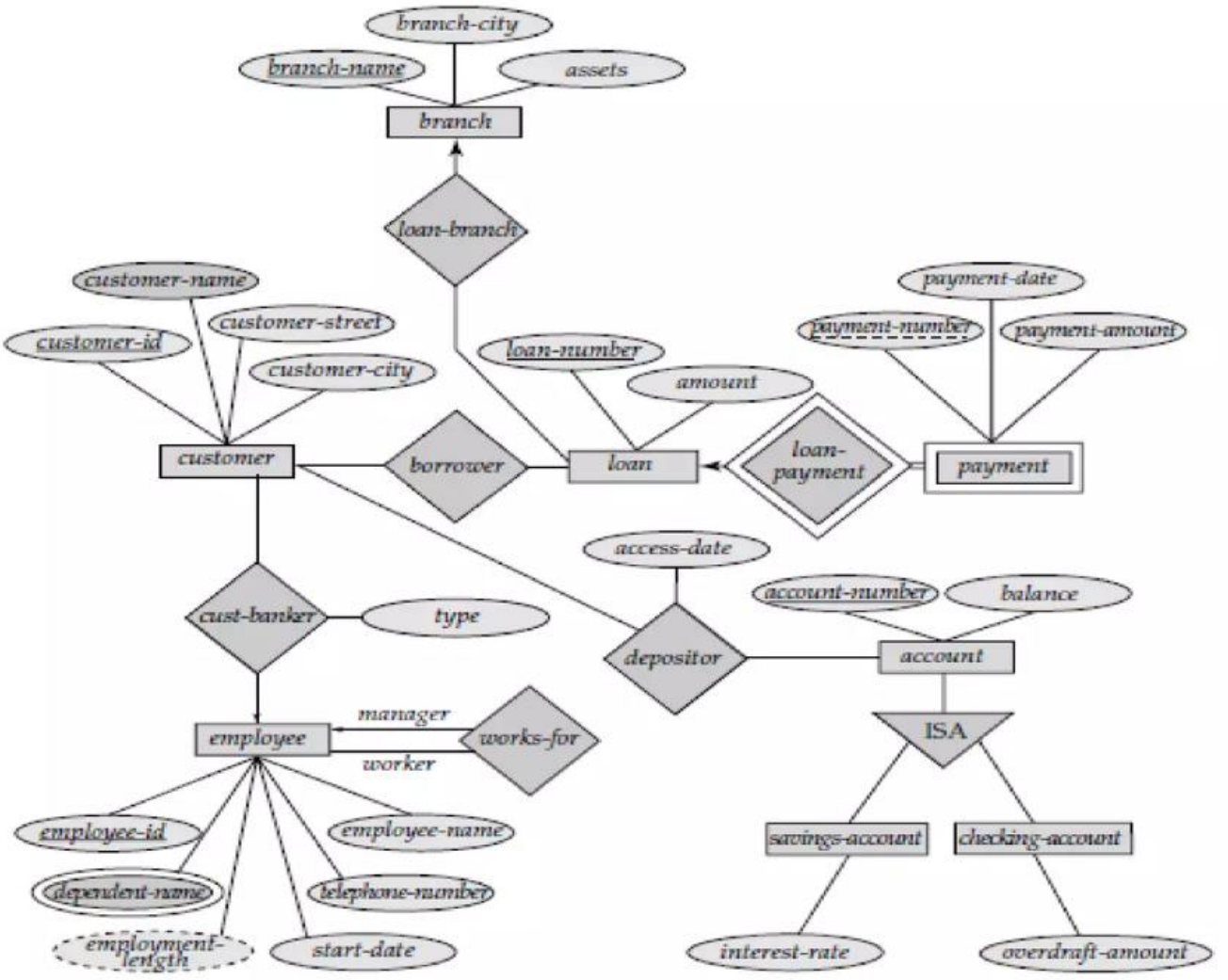
With **Aggregation** (without introducing redundancy) the ER diagram can be represented as:

- An employee works on a particular job at a particular branch
- An employee, branch, job combination may have an associated manager



ER Diagram with Aggregation

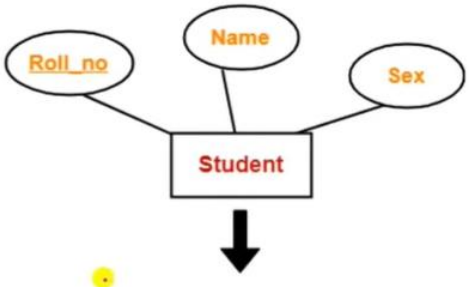
Draw EER diagram on baking system



Conversion from ER diagram to relational schema

Rule1: Strong Entity Set With Only Simple Attributes

- A strong entity set with only ***simple attributes*** will require only **one table** in relational model.
  - Attributes of the table will be the attributes of the entity set.
  - The primary key of the table will be the key attribute of the entity set.

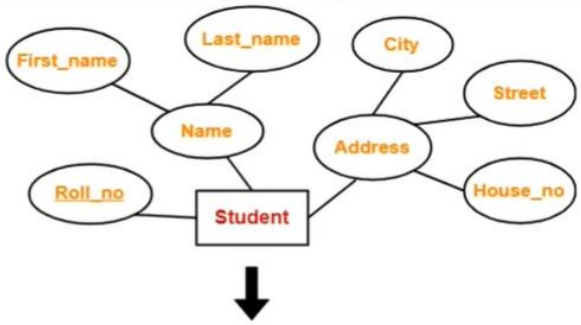


Roll_no	Name	Sex

Schema : Student (Roll_no , Name , Sex)

Rule 2: Strong Entity Set With Composite Attributes

- A strong entity set with any number of ***composite attributes*** will require **only one table** in relational model.
- While conversion, simple attributes of the composite attributes are taken into account and not the composite attribute itself.

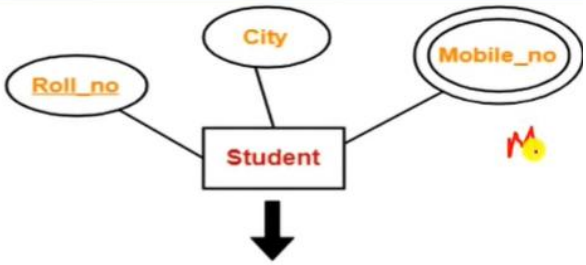


Roll_no	First_name	Last_name	House_no	Street	City

Schema : Student (Roll_no , First_name , Last_name , House_no , Street , City)

Rule 3: Strong Entity Set With Multi Valued Attributes

- A strong entity set with any number of ***multi-valued attributes*** will require **two tables** in relational model.
  - One table will contain all the simple attributes with the primary key.
  - Other table will contain the primary key and all the multi valued attributes.

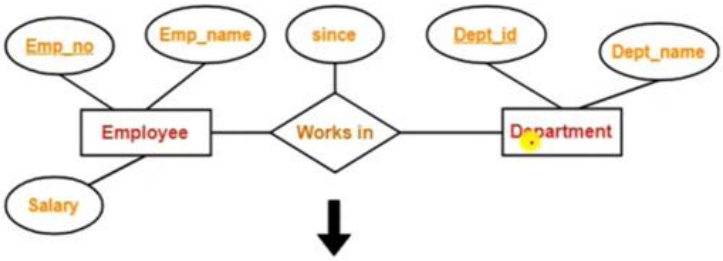


Roll_no	City

Roll_no	Mobile_no

Rule 4: Translating Relationship Set into a Table

- A ***relationship set*** will require **one table** in the relational model.
- Attributes of the table are:
  - Primary key attributes of the participating entity sets
  - Its own descriptive attributes if any.
- Set of non-descriptive attributes will be the primary key
- For given ER diagram, three tables will be required in relational model-
  - One table for the entity set "Employee"
  - One table for the entity set "Department"
  - One table for the relationship set "Works in"



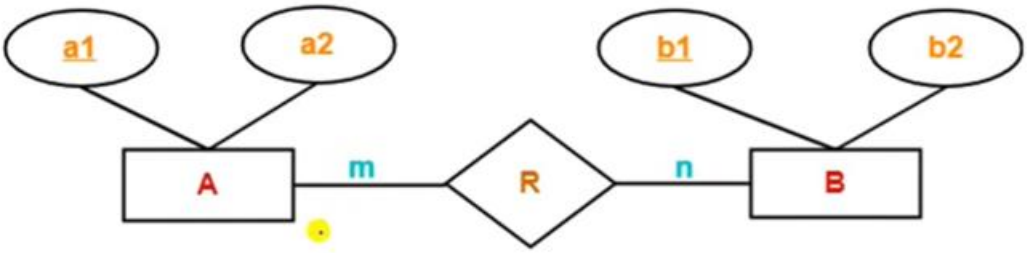
Emp_no	Dept_id	since

Schema : Works in (Emp_no , Dept_id , since)

Rule 5: For Binary Relationships With Cardinality Ratios

- The following four cases are possible-
  - **Case-1:** Binary relationship with cardinality ratio m:n
  - **Case-2:** Binary relationship with cardinality ratio 1:n
  - **Case-3:** Binary relationship with cardinality ratio m:1
  - **Case-4:** Binary relationship with cardinality ratio 1:1

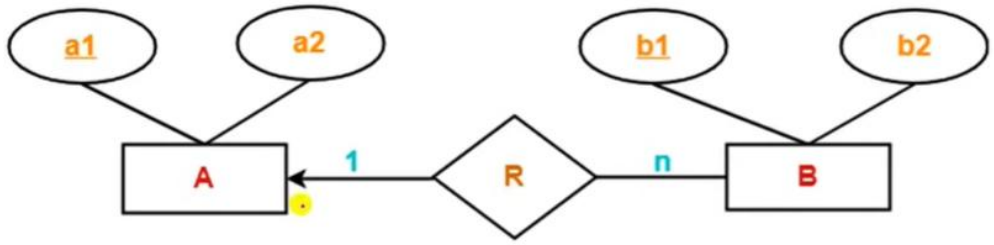
Case-1: For Binary Relationship with Cardinality Ratio m:n



In ***Many-to-Many relationship***, **three tables** will be required-

1. **A (a1 , a2)**
2. **R (a1 , b1)**
3. **B (b1 , b2)**

Case-2: For Binary Relationship with Cardinality Ratio 1:n

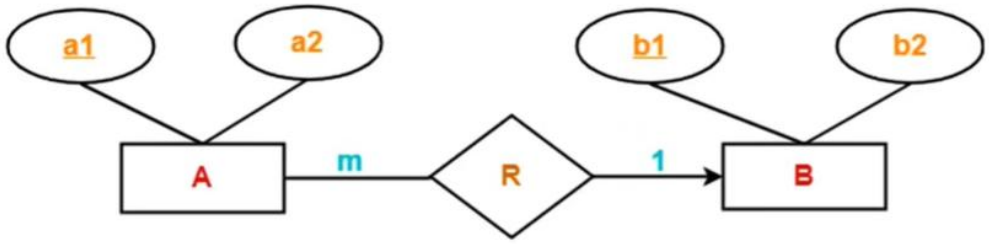


In ***One-to-Many relationship*, two tables** will be required-

1. **A (a1 , a2)**
2. **BR (b1 , b2, a1)**

NOTE - Here, combined table will be drawn for the entity set B and relationship set R.

Case-3: For Binary Relationship with Cardinality Ratio m:1

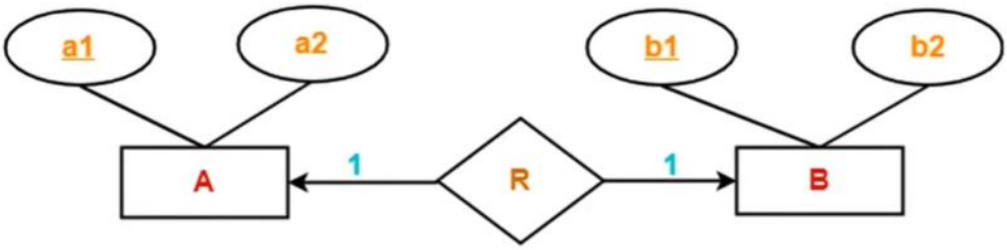


In ***Many-to-One relationship*, two tables** will be required-

1. **AR (a1 , a2 , b1)**
2. **B (b1 , b2)**

NOTE - Here, combined table will be drawn for the entity set A and relationship set R.

Case-4: For Binary Relationship with Cardinality Ratio 1:1



In ***One-to-One relationship***, **two tables** will be required. Either combine 'R' with 'A' or 'B'

Way-01:

1. AR (**a1** , a2 , b1)
2. B (**b1** , b2)

Way-02:

1. A (**a1** , a2)
2. BR (**b1** , b2, a1)

Thumb Rules to Remember:

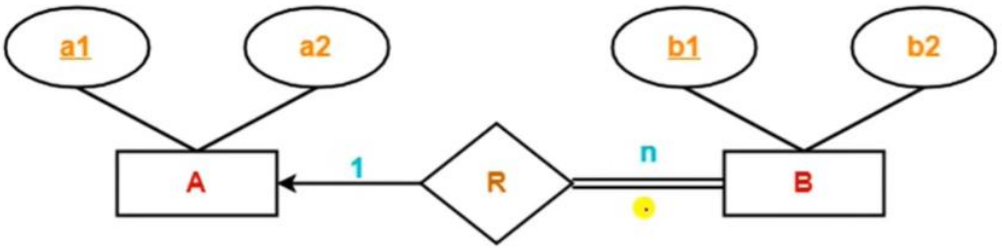
(For determining minimum number of tables)

- **While determining the minimum number of tables required for binary relationships with given cardinality ratios**, following thumb rules must be kept in mind-
  - For binary relationship with cardinality ration **m : n** ,
    - *Separate and individual tables* will be drawn for each entity set and relationship (i.e. **three tables** will be required).
  - For binary relationship with cardinality ratio either **m : 1 or 1 : n** ,
    - Always remember "many side will consume the relationship" i.e. a combined table will be drawn for many side entity set and relationship set (i.e. **Two tables** will be required).
  - For binary relationship with cardinality ratio **1 : 1** ,
    - **Two tables** will be required. You can combine the relationship set with any one of the entity sets.

Rule-6: For Binary Relationship with Both Cardinality Constraint and Participation Constraints

- Cardinality constraints will be implemented as discussed in Rule-5.
- Because of the *total participation constraint*, foreign key acquires **NOT NULL** constraint i.e. now foreign key can not be null.
- Two Cases:
  - **Case-1:** For Binary Relationship with Cardinality Constraint and Total Participation Constraint from One Side
  - **Case-2:** For Binary Relationship with Cardinality Constraint and Total Participation Constraint from Both Sides

Case-1: For Binary Relationship with Cardinality Constraint and Total Participation Constraint from One Side



Because cardinality ratio = 1 : n , we will combine the entity set B and relationship set R.

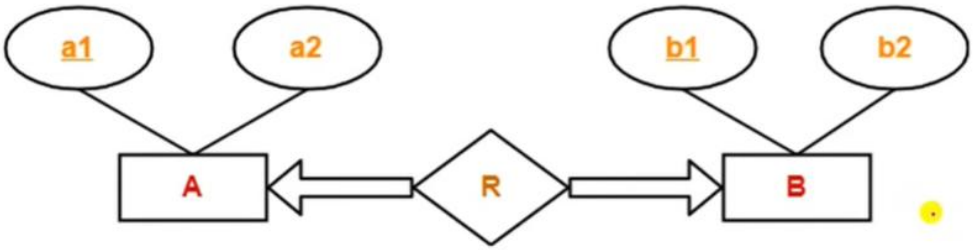
Then, **two tables** will be required-

1. **A (a1 , a2)**
2. **BR (b1 , b2 , a1)**

Because of total participation, ***foreign key a1*** has acquired ***NOT NULL constraint***, so it can't be null now.

Case-2: For Binary Relationship with Cardinality Constraint and Total Participation Constraint from Both Sides

- If there is a key constraint from both the sides of an entity set with total participation, then that binary relationship is represented using **only single table.**

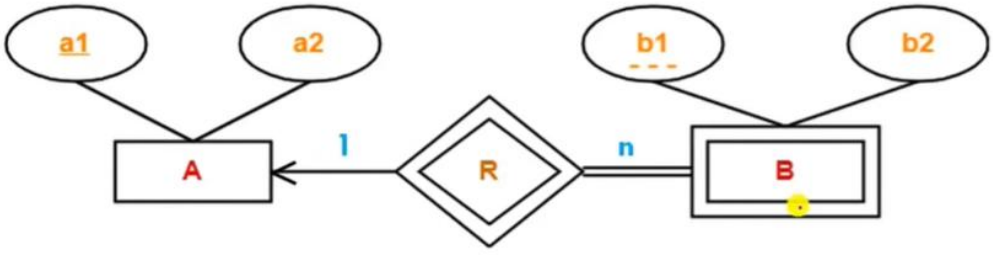


- Here, Only **one table** is required.

1. **ARB (a1 , a2 , b1 , b2)**

Rule-7: For Binary Relationship with Weak Entity Set

- Weak entity set always appears in association with identifying relationship with **total** participation constraint and there is always **1: n** relationship from identifying entity set to weak entity set.



- Here, **two tables** will be required-

1. **A (a1 , a2)**
2. **BR (a1 , b1 , b2)**